

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 12 - SEARCHING



MUHAMAD MIRZA AGUSTA

109082500044

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Praktikum Modul 12 ini berfokus pada konsep algoritma pencarian (searching) dalam pemrograman menggunakan bahasa Go. Searching adalah proses menemukan lokasi atau keberadaan suatu elemen tertentu di dalam kumpulan data. Dua algoritma pencarian yang dipelajari dalam praktikum ini adalah Sequential Search (Linear Search) dan Binary Search, yang masing-masing memiliki karakteristik, kelebihan, dan kekurangan tersendiri.

A. Sequential Search (Pencarian Sekuensial)

Sequential search adalah algoritma pencarian paling sederhana yang bekerja dengan memeriksa setiap elemen dalam struktur data satu per satu secara berurutan dari indeks pertama hingga terakhir. Algoritma ini membandingkan setiap elemen dengan nilai yang dicari menggunakan operator perbandingan. Jika ditemukan kecocokan, pencarian dihentikan dan indeks elemen tersebut dikembalikan. Jika seluruh elemen telah diperiksa dan tidak ditemukan kecocokan, maka fungsi mengembalikan nilai -1 yang menandakan data tidak ditemukan. Kompleksitas waktu sequential search adalah $O(n)$ dalam kasus terburuk, di mana n adalah jumlah elemen data.

B. Binary Search (Pencarian Biner)

Binary search adalah algoritma pencarian yang jauh lebih efisien dibandingkan sequential search, namun memiliki syarat utama yaitu data harus dalam keadaan terurut (baik ascending maupun descending). Prinsip kerja binary search adalah membagi ruang pencarian menjadi dua bagian secara berulang (divide and conquer). Algoritma ini bekerja dengan menentukan batas kiri (low) dan batas kanan (high) dari array, kemudian menghitung indeks tengah (mid). Nilai pada indeks tengah dibandingkan dengan nilai yang dicari. Jika sama, pencarian selesai. Jika nilai yang dicari lebih besar, maka pencarian dilanjutkan di bagian kanan (mid+1 sampai high). Jika lebih kecil, pencarian dilanjutkan di bagian kiri (low sampai mid-1). Proses ini berulang hingga nilai ditemukan atau batas kiri melampaui batas kanan.

C. Pencarian pada Data Terstruktur (Struct)

Dalam pemrograman nyata, data seringkali tidak berdiri sendiri sebagai tipe sederhana seperti integer atau string, tetapi sebagai struktur data yang memiliki banyak field. Pencarian pada data terstruktur dapat dilakukan dengan membandingkan salah satu field yang dijadikan sebagai kunci pencarian (key). Contohnya dalam program pengelolaan data mahasiswa, pencarian dapat dilakukan berdasarkan field NIM, sementara field nama dan IPK ikut ditampilkan ketika data ditemukan. Algoritma sequential search maupun binary search dapat diadaptasi untuk bekerja pada array of struct dengan cara membandingkan field yang sesuai.

D. Pencarian dengan Data Duplikat

Dalam situasi di mana data mengandung nilai duplikat, seringkali diperlukan pencarian posisi pertama (paling kiri) dari nilai tersebut dalam array yang terurut. Binary search standar akan mengembalikan salah satu indeks dari nilai yang ditemukan, tetapi belum tentu indeks yang paling kiri. Oleh karena itu, binary search perlu dimodifikasi dengan melakukan perulangan ke kiri setelah nilai ditemukan untuk menemukan indeks

pertama dari nilai duplikat. Modifikasi ini mempertahankan kompleksitas $O(\log n)$ untuk pencarian, ditambah $O(k)$ untuk traversing duplikat di mana k adalah jumlah duplikat.

E. Pencarian Dua Nilai Terbesar (Top Two)

Dalam aplikasi seperti pemilihan suara, seringkali diperlukan tidak hanya pemenang pertama (nilai tertinggi), tetapi juga pemenang kedua (nilai tertinggi kedua). Algoritma pencarian dua nilai terbesar dapat diimplementasikan dengan melakukan satu kali perulangan sambil mempertahankan dua variabel untuk menyimpan nilai tertinggi dan tertinggi kedua beserta identitasnya. Jika ditemukan nilai baru yang lebih besar dari nilai tertinggi saat ini, maka nilai tertinggi kedua diperbarui dengan nilai tertinggi sebelumnya, dan nilai tertinggi diperbarui dengan nilai baru. Jika ditemukan nilai yang berada di antara tertinggi kedua dan tertinggi, maka nilai tertinggi kedua diperbarui.

B. Guided

1. [Program Pencarian Data Binatang dengan Sequential Search]

```
package main

import "fmt"

type arrdata [5]string // tipe data alias

func seqsearch(arr arrdata, binatangcari string) int {
    for i := 0; i < len(arr); i++ {
        if arr[i] == binatangcari {
            return i
        }
    }
    return -1
}

func main() {
    var arrbinatang arrdata

    for i := 0; i < len(arrbinatang); i++ {
        fmt.Printf("Masukkan data binatang ke-%d : ", i)
        fmt.Scan(&arrbinatang[i])
    }

    fmt.Println()

    var binatangcari string
    fmt.Print("Masukkan nama binatang yang ingin dicari : ")
    fmt.Scan(&binatangcari)

    idxcari := seqsearch(arrbinatang, binatangcari)

    if idxcari != -1 {
        fmt.Printf("Binatang %s ditemukan pada index ke-%d\n",
            binatangcari, idxcari)
    } else {
        fmt.Printf("Binatang %s tidak ditemukan dalam array\n",
            binatangcari)
    }
}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\week 11\guided\nol\tempCodeRunnerFile.go"
Masukkan data binatang ke-0 : ayam
Masukkan data binatang ke-1 : bebek
Masukkan data binatang ke-2 : jerapah
Masukkan data binatang ke-3 : hiu
Masukkan data binatang ke-4 : singa

Masukkan nama binatang yang ingin dicari : hiu
Binatang hiu ditemukan pada index ke-3
```

Penjelasan :

Program ini dideklarasikan dengan tipe data alias `arrdata` yang merupakan array berukuran 5 elemen bertipe string, berfungsi sebagai wadah untuk menyimpan nama-nama binatang. Program ini memiliki fungsi utama `seqsearch` yang menerima dua parameter yaitu array `arr` bertipe `arrdata` dan `binatangcari` bertipe string, kemudian mengembalikan indeks posisi binatang jika ditemukan atau nilai -1 jika tidak ditemukan. Algoritma sequential search bekerja dengan melakukan perulangan dari indeks 0 hingga panjang array, membandingkan setiap elemen array dengan nilai yang dicari menggunakan operator `==`, dan segera mengembalikan indeks ketika ditemukan kecocokan. Dalam fungsi `main`, program meminta pengguna untuk memasukkan 5 nama binatang satu per satu menggunakan perulangan `for`, kemudian menyimpannya ke dalam array `arrbinatang`. Setelah semua data terisi, program meminta pengguna memasukkan nama binatang yang ingin dicari, lalu memanggil fungsi `seqsearch` untuk melakukan pencarian. Hasil pencarian ditampilkan dengan format yang informatif, yaitu menampilkan indeks jika ditemukan atau pesan tidak ditemukan jika hasil pencarian mengembalikan -1.

2. [Program Pencarian Data dengan Algoritma Binary Search]

```
package main

import "fmt"

// fungsi binary search
func binarySearch(data []int, x int) int {

    // batas kiri array
    kiri := 0

    // batas kanan array
    kanan := len(data) - 1

    // perulangan selama kiri <= kanan
    for kiri <= kanan {
```

```
// mencari index tengah
tengah := (kiri + kanan) / 2

// jika data ditemukan
if data[tengah] == x {
return tengah

// jika x lebih besar, cari ke kanan
} else if data[tengah] < x {
kiri = tengah + 1

// jika x lebih kecil, cari ke kiri
} else {
kanan = tengah - 1
}
}

// jika data tidak ditemukan
return -1
}

func main() {

var n int
var cari int
```

```
// input jumlah data
fmt.Print("Jumlah data: ")
fmt.Scan(&n)

// membuat array sesuai jumlah data
data := make([]int, n)

// input isi array
fmt.Println("Masukkan data terurut:")
for i := 0; i < n; i++ {
    fmt.Scan(&data[i])
}

// input angka yang dicari
fmt.Print("Angka yang dicari: ")
fmt.Scan(&cari)

// memanggil fungsi binary search
hasil := binarySearch(data, cari)

// output hasil pencarian
if hasil != -1 {
    fmt.Println("Data ditemukan di index:", hasil)
} else {
```

```
fmt.Println("Data tidak ditemukan")

}

}
```

Output :

```
PS C:\Users\M S I\10908250044_MUHAMAD-MIRZA-AGUSTA\10908250044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I\10908250044_MIRZA-AGUSTA\week 11\guided\no2\tempCodeRunnerFile.go"
Jumlah data: 4
Masukkan data terurut:
3 5 6 9
Angka yang dicari: 6
Data ditemukan di index: 2
```

Penjelasan :

Program ini dideklarasikan dengan fungsi binary Search yang menerima dua parameter yaitu slice of integer data dan nilai integer x yang akan dicari, kemudian mengembalikan indeks posisi nilai tersebut jika ditemukan atau -1 jika tidak ditemukan. Algoritma binary search bekerja dengan membagi ruang pencarian menjadi dua bagian secara berulang. Fungsi ini menginisialisasi batas kiri dengan nilai 0 dan batas kanan dengan nilai panjang slice dikurangi satu, kemudian melakukan perulangan selama kiri <= kanan. Pada setiap iterasi, program menghitung indeks tengah sebagai $(kiri+kanan)/2$, kemudian membandingkan nilai data[tengah] dengan nilai yang dicari. Jika sama, fungsi mengembalikan indeks tengah. Jika nilai yang dicari lebih besar dari data[tengah], maka batas kiri digeser ke tengah+1 untuk mencari di bagian kanan. Jika lebih kecil, batas kanan digeser ke tengah-1 untuk mencari di bagian kiri. Proses ini terus berulang hingga nilai ditemukan atau batas kiri melampaui kanan yang menandakan data tidak ada. Hasil ditampilkan dengan format indeks jika ditemukan atau pesan tidak ditemukan jika mengembalikan -1.

3. [Program Binary Search untuk Pencarian Data pada Array Terurut]

```
package main

import "fmt"

// fungsi binary search

func binarySearch(data []int, x int) int {

// batas kiri array
```



```
kiri := 0

// batas kanan array
kanan := len(data) - 1

// perulangan selama kiri <= kanan
for kiri <= kanan {

    // mencari index tengah
    tengah := (kiri + kanan) / 2

    // jika data ditemukan
    if data[tengah] == x {
        return tengah
    }

    // jika x lebih besar, cari ke kanan
    } else if data[tengah] < x {
        kiri = tengah + 1
    }

    // jika x lebih kecil, cari ke kiri
    } else {
        kanan = tengah - 1
    }
}
```

```
// jika data tidak ditemukan
return -1
}

func main() {

var n int
var cari int

// input jumlah data
fmt.Print("Jumlah data: ")
fmt.Scan(&n)

// membuat array sesuai jumlah data
data := make([]int, n)

// input isi array
fmt.Println("Masukkan data terurut:")
for i := 0; i < n; i++ {
    fmt.Scan(&data[i])
}

// input angka yang dicari
fmt.Print("Angka yang dicari: ")
fmt.Scan(&cari)
```

```
// memanggil fungsi binary search

hasil := binarySearch(data, cari)


// output hasil pencarian

if hasil != -1 {

fmt.Println("Data ditemukan di index:", hasil)

} else {

fmt.Println("Data tidak ditemukan")

}

}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA\week 11\guided\no3\3.go"
--- MASUKKAN DATA NIM MAHASISWA SECARA TERURUT ---
=== Masukkan data mahasiswa 06 pada indeks ke-0 ===
Masukkan nama : ronaldo
Masukkan NIM : 109082500007
Masukkan IPK : 3.99
-----
=== Masukkan data mahasiswa 06 pada indeks ke-1 ===
Masukkan nama : messi
Masukkan NIM : 109082500008
Masukkan IPK : 4.00
-----
=== Masukkan data mahasiswa 06 pada indeks ke-2 ===
Masukkan nama : piero
Masukkan NIM : 109082500002
Masukkan IPK : 3.67
-----
=== Masukkan data mahasiswa 06 pada indeks ke-3 ===
Masukkan nama : zico
Masukkan NIM : 109082500005
Masukkan IPK : 3.87
-----
=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan NIM mahasiswa yang ingin dicari : 109082500002 piero

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 109082500002 ditemukan pada array di indeks ke-2!
Berikut Detail Datanya :
Nama : piero
NIM : 109082500002
IPK : 3.67

== HASIL BINARY SEARCH ==
Data NIM mahasiswa 109082500002 ditemukan pada array di indeks ke-2!
Berikut Detail Datanya :
Nama : piero
NIM : 109082500002
IPK : 3.67
```

Penjelasan :

Program mendefinisikan struct mahasiswa dengan tiga field yaitu nama bertipe string, nim bertipe integer, dan IPK bertipe float64. Selanjutnya didefinisikan tipe arrMahasiswa sebagai array berukuran 5 elemen bertipe mahasiswa yang berfungsi sebagai wadah untuk menyimpan data mahasiswa. yaitu seqSearchStruct untuk sequential search dan binSearchStruct untuk binary

Fungsi binSearchStruct bekerja dengan algoritma binary search yang membagi ruang pencarian menjadi dua bagian secara berulang. Fungsi ini menginisialisasi batas kiri (kr) dengan 0 dan batas kanan (kn) dengan panjang array dikurangi satu, kemudian melakukan perulangan selama $kr \leq kn$ dan found masih -1. Pada setiap iterasi, program menghitung indeks tengah (med), lalu membandingkan NIM yang dicari dengan arrData[med].nim. Jika lebih kecil, batas kanan digeser ke med-1. Jika lebih besar, batas kiri digeser ke med+1. Jika sama, nilai found diisi dengan indeks tengah.

Dalam fungsi main, program meminta pengguna memasukkan data 5 mahasiswa secara berurutan dengan NIM yang harus dimasukkan dalam keadaan sudah terurut agar binary search dapat berfungsi dengan benar. Setelah semua data terisi, program meminta NIM yang ingin dicari, kemudian memanggil kedua fungsi pencarian secara bergantian. Hasil pencarian ditampilkan dengan format yang informatif, mencakup indeks ditemukan serta detail data mahasiswa seperti nama, NIM, dan IPK dengan format dua desimal jika ditemukan, atau pesan tidak ditemukan jika hasil pencarian mengembalikan -1.

C. Unguided

1. Soal Unguided

Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara dapat segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau dengan nomor pilihan di luar yang tersedia, sehingga data juga harus divalidasi. Tugas Anda untuk membuat program mencari siapa yang memenangkan pemilihan ketua RT

Jawaban :

```
package main

import "fmt"

func main() {
    var input int
    votes := make(map[int]int)
    validCount := 0
    totalCount := 0

    for {
        fmt.Scan(&input)

        if input == 0 {
            break
        }

        totalCount++

        if input >= 1 && input <= 20 {
            validCount++
            votes[input]++
        }
    }

    fmt.Printf("Suara masuk: %d\n", totalCount)
    fmt.Printf("Suara sah: %d\n", validCount)

    for i := 1; i <= 20; i++ {
        if votes[i] > 0 {
            fmt.Printf("%d: %d\n", i, votes[i])
        }
    }
}
```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\week 11\unguided\n01\1.go"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2
```

Penjelasan :

Program ini dirancang untuk menerima input bilangan bulat dalam rentang 1 hingga 20 secara terus menerus hingga pengguna memasukkan angka 0 sebagai tanda berakhirnya input. Program mendeklarasikan map votes untuk menyimpan jumlah suara setiap kandidat, serta variabel validCount dan total Count untuk menghitung suara sah dan total suara. Di dalam perulangan, program membaca input, dan jika input bernilai 0 maka perulangan berhenti. Untuk setiap input, total Count bertambah, dan jika input berada dalam rentang 1 hingga 20, maka validCount bertambah dan map votes diperbarui dengan menambahkan satu pada key yang sesuai. Setelah perulangan berakhir, program menampilkan total suara masuk, total suara sah, kemudian melakukan perulangan dari 1 hingga 20 untuk menampilkan nomor pilihan beserta jumlah suaranya jika nilai pada map lebih dari 0. Program ini merupakan implementasi yang baik untuk memahami konsep map sebagai penghitung frekuensi, validasi input, perulangan dengan kondisi break.

2. Soal Unguided

Berdasarkan program sebelumnya, buat program pilkart yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang sama, ketua terpilih adalah dengan nomor peserta yang paling kecil dan wakilnya dengan nomor peserta terkecil berikutnya.

Jawaban :

```
package main

import "fmt"

func main() {
    var input int
    votes := make(map[int]int)
    validCount := 0
    totalCount := 0
```

```

for {
    fmt.Scan(&input)

    if input == 0 {
        break
    }

    totalCount++

    if input >= 1 && input <= 20 {
        validCount++
        votes[input]++
    }

}

fmt.Printf("Suara masuk: %d\n", totalCount)
fmt.Printf("Suara sah: %d\n", validCount)

firstCandidate := 0
firstVotes := -1
secondCandidate := 0
secondVotes := -1

for i := 1; i <= 20; i++ {
    if votes[i] > 0 {
        if votes[i] > firstVotes {

            secondVotes = firstVotes
            secondCandidate = firstCandidate

            firstVotes = votes[i]
            firstCandidate = i
        } else if votes[i] > secondVotes {

            secondVotes = votes[i]
            secondCandidate = i
        } else if votes[i] == firstVotes {

            if i < firstCandidate {

                secondVotes = firstVotes
                secondCandidate = firstCandidate

                firstCandidate = i
            } else if i < secondCandidate ||
secondCandidate == 0 {

                secondCandidate = i
            }

```

```

        } else if votes[i] == secondVotes {

            if i < secondCandidate {
                secondCandidate = i
            }
        }
    }

    fmt.Printf("Ketua RT: %d\n", firstCandidate)
    fmt.Printf("Wakil ketua: %d\n", secondCandidate)
}

```

Output :

```

PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run
MIRZA-AGUSTA\week 11\unguided\no2\no2.go
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19

```

Penjelasan :

Program ini adalah aplikasi penghitungan suara untuk pemilihan Ketua RT dan Wakil Ketua berdasarkan suara yang masuk menggunakan map dalam bahasa Go. Program menerima input bilangan bulat dari 1 hingga 20 secara terus menerus hingga angka 0 dimasukkan sebagai tanda berhenti. Setelah menghitung total suara masuk dan suara sah, program mencari dua kandidat dengan suara tertinggi untuk ditetapkan sebagai Ketua dan Wakil Ketua, dengan aturan bahwa jika terdapat suara yang sama, kandidat dengan nomor lebih kecil (indeks lebih rendah) yang diprioritaskan. Program menggunakan variabel firstCandidate dan firstVotes untuk menyimpan suara tertinggi, serta secondCandidate dan secondVotes untuk suara tertinggi kedua. Saat melakukan perulangan dari 1 hingga 20, program membandingkan setiap suara dan memperbarui posisi pertama dan kedua dengan logika yang mempertimbangkan nilai suara maupun nomor kandidat saat terjadi kondisi seri. Program ini merupakan implementasi yang baik untuk memahami konsep map sebagai penghitung frekuensi.

3. Soal Unguided

Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k, apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA"

Jawaban :

```

package main

```



```

import "fmt"

const NMAX = 1000000

var data [NMAX]int

func main() {
    var n, k int

    fmt.Scan(&n, &k)

    isiArray(n)

    pos := posisi(n, k)

    if pos == -1 {
        fmt.Println("TIDAK ADA")
    } else {
        fmt.Println(pos)
    }
}

func isiArray(n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {

    kiri := 0
    kanan := n - 1

    for kiri <= kanan {
        tengah := (kiri + kanan) / 2

        if data[tengah] == k {

            pos := tengah
            for pos > 0 && data[pos-1] == k {
                pos--
            }
            return pos
        } else if data[tengah] < k {
            kiri = tengah + 1
        } else {
            kanan = tengah - 1
        }
    }

    return -1
}

```

Output :

```
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\MIRZA-AGUSTA\week 11\unguided\no3\tempCodeRunnerFile.go"
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8
PS C:\Users\M S I\109082500044_MUHAMAD-MIRZA-AGUSTA\109082500044_MUHAMAD-MIRZA-AGUSTA> go run "c:\MIRZA-AGUSTA\week 11\unguided\no3\tempCodeRunnerFile.go"
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA
```

Penjelasan :

Program ini adalah aplikasi pencarian data menggunakan algoritma binary search yang dimodifikasi untuk menemukan posisi pertama (paling kiri) dari suatu nilai dalam array yang sudah terurut. Program dideklarasikan dengan konstanta NMAX sebesar 1.000.000 sebagai kapasitas maksimal array, serta variabel global data sebagai array bertipe integer dengan ukuran tersebut. Program membaca dua input yaitu n untuk jumlah data dan k untuk nilai yang akan dicari, kemudian memanggil fungsi isi Array untuk mengisi n buah bilangan bulat ke dalam array. Fungsi posisi mengimplementasikan binary search untuk mencari nilai k, dengan keunikan bahwa ketika nilai ditemukan pada indeks tengah, program tidak langsung mengembalikan indeks tersebut melainkan melakukan perulangan ke kiri sebanyak mungkin (for pos > 0 && data[pos-1] == k) untuk menemukan indeks paling kiri dari nilai yang sama. Hal ini penting karena array mungkin berisi nilai duplikat, dan soal kemungkinan meminta posisi pertama kemunculan. Jika nilai ditemukan, fungsi mengembalikan indeks posisi pertama; jika tidak ditemukan, fungsi mengembalikan -1.

D. Kesimpulan

Berdasarkan praktikum Modul 11 tentang searching (pencarian data) ini, dapat disimpulkan bahwa algoritma pencarian merupakan fondasi penting dalam pengolahan data yang memungkinkan program untuk menemukan dan mengakses informasi secara efisien. Melalui serangkaian program guided dan unguided, praktikum ini berhasil mendemonstrasikan dua algoritma pencarian utama yaitu sequential search dan binary search, masing-masing dengan karakteristik dan kasus penggunaan yang berbeda. Sequential search diimplementasikan pada program pencarian nama binatang, di mana algoritma ini cocok digunakan karena data tidak harus terurut dan jumlah data kecil (hanya 5 elemen), bekerja dengan memeriksa setiap elemen satu per satu dari indeks 0 hingga akhir. Sementara itu, binary search diimplementasikan pada program pencarian angka dalam data terurut, di mana algoritma ini mampu mencari data dengan kompleksitas $O(\log n)$ yang sangat efisien untuk ukuran data besar dengan cara membagi ruang pencarian menjadi dua bagian secara berulang. Program guided ketiga mengembangkan pencarian pada data terstruktur (struct mahasiswa) dengan field NIM sebagai kunci pencarian, mengimplementasikan kedua algoritma sekaligus untuk dibandingkan, serta menampilkan detail data mahasiswa ketika ditemukan, menunjukkan bahwa algoritma pencarian dapat diadaptasi untuk berbagai tipe data.

Pada unguided pertama dan kedua, program penghitungan suara pemilihan ketua RT berhasil mengimplementasikan map sebagai penghitung frekuensi untuk mencatat jumlah suara 20 calon, kemudian dikembangkan dengan logika pencarian dua nilai terbesar untuk menentukan Ketua RT (perolehan suara tertinggi) dan Wakil Ketua (perolehan suara tertinggi kedua). Program ini menangani kondisi tie (suara sama) dengan memprioritaskan calon bernomor lebih kecil, diimplementasikan melalui perbandingan nilai suara dan nomor calon secara cermat dalam satu kali perulangan. Pada unguided ketiga, program pencarian menggunakan binary search yang dimodifikasi untuk menemukan posisi pertama (paling kiri) dari suatu nilai dalam array terurut yang mengandung duplikat, dengan menambahkan perulangan ke kiri setelah nilai ditemukan. Secara keseluruhan, praktikum Modul 11 ini memberikan pemahaman yang mendalam tentang perbedaan kompleksitas waktu antara sequential search ($O(n)$) dan binary search ($O(\log n)$), pentingnya data terurut untuk binary search, adaptasi algoritma untuk data terstruktur, penggunaan map sebagai penghitung frekuensi, pencarian dua nilai terbesar dengan penanganan tie, serta modifikasi binary search untuk data duplikat. Pemahaman ini akan sangat berguna dalam pengembangan aplikasi yang membutuhkan akses data secara cepat dan efisien.